gem5 MCP Server

Driving a computer-architecture simulator from an AI assistant

This document explains what you built: a Model Context Protocol (MCP) server that lets an LLM (via Claude Desktop) run and analyse gem5 simulations. gem5 is a cycle-level simulator of CPUs, caches and memory used in computer-architecture research. It has no programming API - you can only launch a compiled binary with a Python config script - so the server wraps that command-line workflow in clean, safe tools an AI can call.

At a glance

Simulator	gem5 v25.1.0.1, built for X86 and RISC-V (SE mode)
Runs on	WSL Ubuntu-24.04, as the non-root 'gem5' user
Server	Python (official MCP SDK), runs on Windows, drives gem5 via WSL
Interface	9 MCP tools exposed to Claude Desktop over stdio
Status	Installed, verified end-to-end, connected and 'running' in Claude

1. The problem it solves

gem5 was never designed to be called by software. Every interaction is a long-running command line that writes results to files. That clashes with how an AI assistant works. The server bridges the gap by handling four hard properties of gem5:

- **No API:** gem5 has no API - it is a CLI binary fed an arbitrary Python script.
- **Slow:** A single run can take minutes to hours, far longer than a tool call should block.
- **Unsafe input:** Config 'scripts' are executable Python - letting a model write them is code execution.
- **Messy output:** Output is one giant flat-text stats file, not structured data.

2. How it works (architecture)

```
Claude Desktop  (Home chat)
      |  MCP over stdio
      v
MCP Server  (Windows Python)
      |  wsl.exe -u gem5 -- gem5.opt <template>.py --outdir=/runs/{job_id}
      v
gem5  (compiled inside WSL Ubuntu: X86 + RISCV)
      |  writes m5out/stats.txt
      v
Stats parser -> filtered JSON        SQLite <- job & checkpoint records
```

The server is essentially a subprocess orchestrator. It launches gem5 inside WSL, gives every job its own output folder, reads the results back from Windows through the \\wsl.localhost\ path, parses them, and returns clean JSON to Claude.

3. The tools Claude can use

Nine tools make up the interface. Claude calls these; you never touch a command line.

list_available_isas	Which ISAs are built and runnable (X86, RISCV).
get_gem5_version	The exact pinned gem5 version.
list_config_templates	The fixed simulation templates + their parameters.
submit_simulation	Launch a run; returns a job_id immediately (non-blocking).
get_job_status	queued / running / done / failed, plus queue position.
get_job_result	Headline summary (cycles, instructions, sim time).
get_stats	Filtered query of the full stats, e.g. pattern 'system.cpu*'.
list_checkpoints	Checkpoints produced by finished jobs.
restore_from_checkpoint	Start a new run resuming from a saved checkpoint.

The job pattern

Because simulations are slow, submit_simulation never blocks. It returns a job_id right away and runs gem5 in the background. Claude then polls get_job_status until the run is 'done', and finally pulls results with get_job_result or get_stats. This keeps the conversation responsive even for long runs.

4. Simulation templates

Instead of accepting raw Python, the server ships a small library of fixed, trusted config scripts. You pick one and supply validated parameters:

- **se_single_core:** Syscall-emulation run of one binary on a single CPU core with L1/L2 caches.
- **se_multi_core:** Same, but on N identical cores (2-16), each with its own cache hierarchy.

Parameters you can set (all validated before anything runs):

- **isa:** X86 or RISCV.
- **cpu:** atomic / timing / o3 / minor - from fast-and-simple to detailed out-of-order.
- **caches:** l1i, l1d and l2 sizes (e.g. 32kB, 256kB, 1MB).
- **system:** memory size, clock frequency, and an optional instruction-count limit.

5. Built-in safety controls

The design deliberately limits what an AI can do with a native simulator on your machine:

- **No code execution:** The model never runs raw Python - only fixed templates, with parameters checked by a strict schema.
- **Bounded pool:** At most 4 simulations run at once; the rest queue, so gem5 can't starve your PC.
- **Caps:** A run is killed if it exceeds 512 MB of output or a 2-hour wall-clock limit.
- **Debug flags:** Verbose trace flags are off by default; only a small safe whitelist is allowed.
- **Isolation:** gem5 runs as a non-root Linux user inside WSL, isolated from Windows.
- **Context safety:** Results are filtered before return - a full stats file (tens of thousands of lines) never floods the chat.

6. How you use it

From Claude Desktop (the main way)

In a normal Home chat, just ask in plain English. For example:

```
- "list the available gem5 ISAs"
- "run a hello-world simulation on X86 (se_single_core,
```

```
   binary /home/gem5/testbins/hello_x86, cpu timing) and show the CPI"
- "run the same binary with an o3 CPU and compare the cycle counts"
```

Standalone (without the AI)

gem5 is a full, normal install you can drive yourself from a WSL shell:

```
wsl -d Ubuntu-24.04 -u gem5
cd ~/gem5
build/X86/gem5.opt configs/deprecated/example/se.py \
    --cmd=/home/gem5/testbins/hello_x86
```

7. Project layout

server/main.py	The MCP server and its 9 tools.
server/jobs.py	Async job queue, launch, size/time caps.
server/gem5_runner.py	Builds the wsl.exe + gem5.opt command.
server/templates.py	Validated template registry (parameter schemas).
server/stats.py	stats.txt -> dict, with glob/regex filtering.
server/db.py	SQLite store for jobs and checkpoints.
templates/*.py	The fixed, trusted gem5 config scripts.
README.md	Setup and Claude Desktop wiring instructions.

Verified working

Both ISAs were tested end-to-end through the server. Example X86 hello-world result: 131,262 instructions, 264,820 cycles, CPI ~2.02, out of 1,139 parsed statistics - all returned to the caller as clean JSON.

Scope note: this version focuses on SE (syscall-emulation) mode, which only needs a compiled binary. Full-system mode (booting a real OS with disk images and kernels) is intentionally left for a future version.